The Frozen Row

How maya and agentty Achieve Scrollback Integrity

in the Terminal

A source-grounded field manual for the inline renderer:
the shadow-of-wire, the Witness Chain, and the state machine
that keeps a streaming TUI from ever corrupting your scrollback.

Derived entirely from the maya and agentty source trees.

Code is the only source of truth.

About this book

This is a book about a single, deceptively narrow engineering problem: how a program that draws in your terminal *inline* — sharing the screen with your shell instead of taking over an alternate buffer — can update that screen thousands of times per second, over a long streaming AI response, without ever corrupting the lines that scroll up out of view into the terminal's native scrollback.

The program is **agentty**, a terminal AI coding assistant written in C++26, and its rendering engine **maya**. The problem sounds small. It is not. Getting it wrong produces the most infuriating class of TUI bug there is: a duplicated composer one screen up, a card cut in half, your shell history silently drifting, or — worst — an entire turn of a conversation stamped permanently into scrollback where it will remain until you close the terminal, because once a row scrolls off the top edge it is *frozen*, owned by the emulator, unreachable by any escape sequence the application can send.

Every claim in this book is drawn from the actual source. Where the source and a comment or a design doc disagree, the source wins. Function names, field names, and the specific escape sequences are quoted as they appear in the tree so you can open the file and check. The relevant files are:

```
maya/include/maya/render/serialize.hpp   InlineFrameState, the witnesses
maya/src/render/serialize.cpp            compose_inline_frame_impl (the diff)
maya/src/render/frame_bytes.cpp          the Witness Chain entrypoint
maya/include/maya/app/app.hpp            Runtime::render + coherence machine
maya/src/render/scrollback_ledger.cpp    seal_measured (freeze accounting)
agentty/src/runtime/app/update/picker.cpp  reset_inline on thread swap
agentty/src/runtime/app/update/frozen.cpp   trim + commit_scrollback
```

The book is meant to be read front to back once, then kept open beside the code. Chapters 1–3 build the mental model of a terminal and state the invariant. Chapters 4–7 are the algorithm. Chapters 8–10 are the type-system armour — the Witness Chain — that makes the invariant un-violatable at compile time. Chapters 11–13 are the recovery state machine and how agentty drives it.

Contents

Chapter 1 — The medium: what a terminal actually is

Before any of the renderer makes sense, you have to hold an accurate model of the machine it is talking to. Almost every scrollback bug in this codebase's history was, at root, a place where the code's model of the terminal diverged from the terminal's real behaviour.

A terminal emulator maintains a grid of character cells — `W` columns wide by some number of rows tall. It has a *cursor* at some (row, column). The application writes a byte stream to a file descriptor; the emulator interprets that stream, mutating cells and moving the cursor. Printable characters write a cell at the cursor and advance the cursor one column right. Control sequences — the ANSI / VT escapes, mostly starting with the byte `0x1b` (ESC) — move the cursor, erase regions, and change colors.

Three behaviours of this machine are load-bearing for everything that follows.

1. There is no read-back of cell contents. The application cannot ask the terminal "what character is at row 4, column 12?" nor "where is the cursor right now?" in any way it can rely on across all terminals. It writes bytes; it never reads the grid. So the renderer cannot *check* its work against the screen. It must *model* the screen perfectly and trust the model.

2. Cursor moves are relative to a viewport whose top drifts. The viewport is the visible window into a taller virtual grid. When the cursor is on the bottom row and a line feed (`\n` arrives, the emulator does not move the cursor down — there is no down — it *scrolls*: every row shifts up one, the top row is pushed out of the viewport, and a fresh blank row appears at the bottom. The cursor stays on the (new) bottom row.

3. A scrolled-off row is frozen. The row that got pushed out is captured into the emulator's *native scrollback* — the buffer you reach by scrolling your mouse wheel up. Its bytes are whatever was on the wire for that row at the instant it scrolled. The application cannot address it any more. There is no cursor position that reaches it. The single escape that can affect it, `\x1b[3J` ("erase saved lines"), can only *delete* the entire scrollback — including your shell history from before the program launched — not selectively rewrite a row.

That third fact is the whole game. The rendering engine gets exactly *one* chance to put the correct bytes on a row: the moment before that row scrolls off. After that, whatever it emitted is permanent. If the engine's model was wrong at that instant, the wrong bytes are frozen into your scrollback forever.

This is why the book is called *The Frozen Row.* The entire architecture exists to guarantee that at the instant any row freezes, the bytes on it are provably correct.

Chapter 2 — Inline mode and the frozen-row problem

A full-screen TUI — vim, htop, less — has it comparatively easy. It switches the terminal into the *alternate screen buffer* (the escape `\x1b[?1049h`), a separate grid that has no scrollback at all. It owns every visible cell. When it exits, the alternate buffer is discarded and your original screen — shell prompt, history — reappears untouched. A full-screen app can repaint any cell at any time because it owns the whole grid and nothing ever scrolls off into a shared buffer.

agentty deliberately does *not* do this. It runs **inline**: it draws below your shell prompt, in the same buffer your shell uses, so that when a conversation grows past the screen it scrolls into the same native scrollback as everything else, and you can scroll up through your whole session — shell commands and AI turns interleaved — as one continuous history. This is a far better user experience. It is also enormously harder to get right, because now the application *shares* the scrollback with the host shell, and every row that scrolls off is a row it can never touch again.

Consider what a streaming AI response does to this arrangement. The model emits text token by token. Each token can grow the rendered content by a fraction of a line to several lines. agentty relays-out and re-renders on every batch of tokens — potentially dozens of frames per second, for a response that ends up hundreds of rows tall. Every one of those frames pushes rows off the top of the viewport into frozen scrollback. If any frame's model is off by a row, the corruption is immediate and permanent.

The failure modes are not hypothetical; they are catalogued in the source and in agentty's own memory of past bugs. Among them:

- The composer (the input box at the bottom) duplicated one screen up, because a shrink parked the physical cursor somewhere the model didn't expect.
- An entire finished turn stamped a duplicate copy into scrollback, because an oversized redraw emitted bottom-edge newlines that scrolled still-visible rows off.
- The user's pre-launch shell history silently *drifting* upward every few frames, because a redraw path pushed host content into scrollback it had no right to touch.
- A card cut in half at the viewport top, because rows already in scrollback could not be rewritten to match a late layout change.

Every one of those is the same underlying event: the renderer's model of the screen drifted from reality, and a wrong row then froze. The rest of this book is the machinery that makes that event impossible.

Chapter 3 — The one invariant: shadow-of-wire

Here is the entire correctness argument in one sentence, quoted from the comment above the debug verifier at the end of `compose_inline_frame_impl` in `serialize.cpp`:

> *"prev_cells[y][x] must, at the end of every compose, equal canvas.cells()[y*W + x] for every (x, y) the wire is about to show — i.e. rows in [updatable_start, content_rows)."*

That is the .**shadow-of-wire**invariant `prev_cells` is the renderer's private buffer holding its belief about what the terminal is currently displaying — its *shadow* of the *wire*. The invariant says: at the end of every frame, the shadow must equal the freshly-laid-out canvas over every cell the viewport shows. Because the next frame computes the bytes to emit as a *diff* between the new canvas and this shadow, the shadow being correct is exactly the precondition for the emitted bytes being correct.

The same comment then states the thesis of the whole design — that every historical corruption bug was a violation of precisely this invariant:

> *"Every historical 'corrupted rows' bug in this file's git log was a version of THIS invariant being temporarily violated: over-commit scrollback shift, cells_max_y heuristic stale, streaming-markdown cache aliasing, empty-CodeBlock ghost, partial-write residue inflation — all of them showed up downstream as either a simd::bulk_eq returning 'unchanged' for a row that did change (frame freezes) or returning 'changed' for a row that didn't (cursor walks redundantly, emitting bytes that drift the wire away from the shadow)."*

Read that carefully, because it defines the two and only two ways the invariant can fail, and therefore the two and only two symptom classes:

False-negative diff
: The diff says a row is unchanged when it actually changed. The renderer emits nothing for it; the screen keeps showing stale content. Symptom: the frame appears *frozen*, not updating.

False-positive diff
: The diff says a row changed when it didn't. The renderer walks the cursor and emits bytes for it — and every emitted byte is an opportunity for the wire to drift away from the shadow, because the emit math assumes a cursor position that may now be wrong. Symptom: cursor drift, and eventually a wrong row freezes.

So the design has exactly one job, stated three equivalent ways:

- Keep the shadow equal to the wire at all times.
- Never let the diff run against a shadow that is not equal to the wire.
- Before any row freezes, guarantee its wire bytes match its shadow.

The next four chapters are how the diff maintains the shadow. The four after that are how the type system makes it impossible to run the diff when the shadow is *not* provably equal to the wire.

Chapter 4 — The shadow: InlineFrameState and prev_cells

The shadow lives in one move-only value type, `InlineFrameState`, declared in `serialize.hpp`. Its type-level annotation states its purpose:

> *"InlineFrameState is the renderer's only handle to its shadow-of-wire — dropping the value loses the model of what the terminal is displaying."*

The storage, verbatim:

```
AlignedBuffer prev_cells_;
int           prev_width_       = 0;
int           prev_rows_        = 0;
bool          decawm_off_       = false;
bool          cursor_hidden_    = false;
int           ghost_rows_above_ = 0;
int           wire_cursor_rows_ = 0;
uint64_t      shadow_hash_      = static_cast<uint64_t>(-1);
uint64_t      gen_              = 0;
std::vector<uint64_t> row_hashes_;
```

Walk the fields, because each one is load-bearing:

`prev_cells_`
: The shadow proper: a packed `prev_rows_` × `prev_width_` array of 64-bit packed cells. This *is* maya's belief about the wire.

`prev_rows_`, `prev_width_`
: Dimensions of the shadow. The renderer *assumes* the physical cursor sits at row `prev_rows_ - 1` — the last row it painted. That assumption is the hinge of the whole diff; every operation that mutates `prev_rows_` must preserve it.

`decawm_off_`, `cursor_hidden_`
: Whether the renderer has already told the terminal to disable autowrap (\x1b[?7l and hide the cursor (\x1b[?25l These modes persist across frames so the escapes are emitted once, not per frame.

`ghost_rows_above_`
: Used only by the recovery path (Chapter 12): the full content height of the frame that was on screen when a soft failure happened, so the recovery emit can walk the correct upward erase distance.

`wire_cursor_rows_`
: `min(content_rows, term_h)` — how many rows of the frame are actually on screen versus scrolled off.

`shadow_hash_`, `row_hashes_`
: The cryptographic-cheap proof that the shadow is intact. One `uint64_t` hash per row, XOR-folded into a single `shadow_hash_`. Chapter 7 is entirely about these.

`gen_`
: A monotonic generation counter, bumped by every content-advancing operation. It stamps scrollback-commit tokens so a stale token from a superseded frame cannot be applied. Chapter 12

shows why this matters.

Move-only, by design. The type deletes its copy constructor and copy assignment:

```
InlineFrameState(const InlineFrameState&)            = delete;
InlineFrameState& operator=(const InlineFrameState&) = delete;
InlineFrameState(InlineFrameState&&) noexcept            = default;
InlineFrameState& operator=(InlineFrameState&&) noexcept = default;
```

The reason is stated at the type: a copy would create *two parallel shadows of the same wire region.* There is exactly one terminal, so there must be exactly one shadow. If you could copy the state, two parts of the program could each believe they own the model of the screen, diff against their own copy, and emit conflicting bytes. Move-only makes the shadow a *linear* resource: it flows from one owner to the next, never duplicated.

Pure advancing operations. Every state transition is an &&-qualified method that consumes `*this` by rvalue and returns the successor by value. You cannot call them on an lvalue; you must write `std::move(state).reset_state()`, and the source `state` is gone afterward. The transitions are:

```
reset_state()            → fresh, capacity retained
finalize()               → emit mode-restore bytes, drop claims
scrollback_marker(rows)  → mint a commit token (const read)
committed(marker)        → shift ‘rows‘ off the top of the shadow
abandoned_for_recovery() → the post-soft-failure shape
```

Because each returns a fresh value and consumes the old, the state machine is effectively pure — `(canvas, state) → (bytes, state′)` — even though the byte-emitter writes into its moved-in local for implementation convenience. This purity is what lets the Witness Chain (Chapters 8–10) reason about correctness with the type system instead of runtime asserts.

Chapter 5 — The diff: compose_inline_frame_impl, step by step

---

The heart of the engine is one function in `serialize.cpp`:

```
std::pair<std::string, InlineFrameState>
compose_inline_frame_impl(const Canvas& canvas,
                          int content_rows,
                          int term_h,
                          const StylePool& pool,
                          InlineFrameState&& state_in,
                          bool synchronized_output,
                          std::string_view reset_prefix);
```

It takes the new canvas and the old shadow, and returns *(bytes to write, new shadow)*. It is a pure transformation in the type system's eyes: it takes the state by &&-move, mutates a function-local copy, returns the successor. Let us walk it.

Step 0 — take ownership, handle degenerate input. It moves the incoming state into a local, and bails immediately on a zero-size canvas. Then, if the terminal width changed since last frame, it resets the state — a width change invalidates every row layout, so the shadow is meaningless and must be rebuilt from scratch:

```
if (state.prev_width_ != W) {
    state = std::move(state).reset_state();
    state.prev_cells_.shrink_to_fit_if_oversized(...);
}
```

Step 1 — compute the diff geometry. Four integers define the frame's relationship to the viewport:

```
const int prev_rows       = state.prev_rows_;
const int prev_on_screen  = std::min(prev_rows, term_h);
const int updatable_start = prev_rows - prev_on_screen;
const int common          = std::min(content_rows, prev_rows);
```

`updatable_start` is the crucial one. If the previous frame was taller than the viewport, some of its top rows already scrolled into frozen scrollback. `updatable_start` is the first row index that is screen*still*on — i.e. still addressable. Rows below it (y < updatable_start are frozen and must never be touched by the diff. This single quantity is what keeps the diff from trying to rewrite the unrewritable.

Step 2 — find the first changed row. Starting at `updatable_start` (never below it), scan down comparing shadow to canvas row by row, using a SIMD bulk equality, and stop at the first difference:

```
int first_changed = common;
if (have_prev) {
    for (int y = updatable_start; y < common; ++y) {
        if (!simd::bulk_eq(cells + y * W, prev + y * W, W)) {
            first_changed = y;
            break;
        }
    }
}
```

On a steady streaming frame only the tail grows, so `first_changed` lands near the bottom and the whole emit is a handful of rows. This is why a growing 14,000-row transcript still costs a few rows of

work per frame, not 14,000.

Step 3 — the no-op early return, and the cursor-hide re-assertion. If nothing changed and the row count is identical, there is nothing to emit — but the function still emits `\x1b[?25l` (hide cursor) first, unconditionally, on every tick. The comment explains why this cannot be latched: anything outside maya can re-show the cursor between frames — a sandboxed subprocess writing to fd 1, an SSH reconnect, a mobile terminal opening its soft keyboard. Since inline mode never uses the hardware cursor (the composer's caret is a painted cell), the real cursor must stay hidden at all times, so the hide is re-asserted every tick, even the idle ones. It is idempotent — six bytes, no flicker.

Step 4 — open the frame, position the cursor. For a normal (non-first) frame, the renderer must move the physical cursor from where it believes it is — row `prev_rows - 1` — to `first_changed`. This is the single most delicate arithmetic in the engine:

```
const int cursor_row_start = prev_rows - 1;
const int delta = first_changed - cursor_row_start;
if (delta < 0) {
    int up = std::min(-delta, prev_on_screen - 1);
    if (up > 0) ansi::write_cursor_up(out, up);
    out += '\r';
} else if (delta == 0) {
    out += '\r';
} else {
    out += "\r\n";              // grow: SCROLLS at bottom edge
    if (delta > 1) ansi::write_cursor_down(out, delta - 1);
}
```

Three cases. Moving *up* (the change is above the last painted row) uses cursor-up, clamped so it can never walk into frozen scrollback. Staying put uses a carriage return. Moving *down* past the previous bottom — i.e. the frame is growing — uses `\r\n`, whose newline scrolls the terminal at the bottom edge. That scroll is intentional and is exactly how an inline frame grows past the viewport. It is also the moment a row freezes, which is why Chapter 6 exists.

Step 5 — the per-row, per-span emit. From `first_changed` down to the last row, the loop advances one row at a time with `\r\n`, and for each changed row finds the changed column span and emits only those cells. Crucially, after emitting a row's cells it *writes those same cells back into the shadow* so the shadow stays equal to the wire:

```
if (is_new_row) {
    std::memcpy(prev_w + y*W, cur_row, W * 8);   // whole new row
} else {
    // only [x_first_diff, x_last_diff+1) could differ
    std::memcpy(prev_w + y*W + lo, cur_row + lo, (hi-lo) * 8);
}
```

This is the shadow-maintenance step. The invariant from Chapter 3 holds inductively: rows before `first_changed` were identical and untouched; changed rows have their changed span copied from canvas into shadow as they are emitted. At loop exit, shadow equals canvas over the whole visible range.

Step 6 — shrink handling. If the new frame is shorter than the old (content_rows < prev_rows rows below the new content still show stale bytes. The renderer re-anchors to column 0, re-emits the bottom row's visible content (idempotent), then erases downward with `\x1b[J`. There is a subtle DECAWM-off correction here: if the bottom row filled to the last column, the cursor is stuck *at* the

right edge (autowrap is off, so it doesn’t advance to a phantom column `W`), and a naive `\x1b[J` would erase the rightmost real cell. The code detects this and instead walks down a row, erases from there, and walks back up — preserving the cursor-row invariant the next frame depends on. This one edge case was the "last column corruption on full-width content" bug (code-block right borders, full-width rules).

The whole function is bracketed, in synchronized-output terminals, by `\x1b[?2026h` and `\x1b[?2026l` so the terminal applies the entire frame atomically — no visible top-to-bottom paint over a slow SSH link.

Chapter 6 — Growing, scrolling, and the will_scroll_off defence

---

Chapter 5's Step 4 and Step 5 both emit `\r\n` at the bottom edge to grow the frame, and each such newline scrolls one row off the top into frozen scrollback. This is the exact instant the frozen-row problem bites: whatever bytes are on the wire for that row become permanent. If the shadow was even slightly stale for that row — a wide-character boundary mis-snap, a layout shift between frames, a subprocess that wrote to the tty — the stale bytes freeze.

The defence is a targeted, unconditional full-row re-emit for exactly the rows about to freeze. From the per-row loop:

```
const int new_visible_top  = std::max(0, content_rows - term_h);
const int prev_visible_top = std::max(0, prev_rows   - term_h);
const bool will_scroll_off =
    (content_rows >= term_h)
    && (y >= prev_visible_top)
    && (y <= new_visible_top);
const bool whole_row =
    will_scroll_off || (full_row && row_first_diff < W);
const int x_first_diff = whole_row
    ? 0
    : snap_first_diff_left(row_first_diff, cur_row, prev_row, W);
```

Read the window carefully. The rows this frame will scroll off are `[prev_visible_top, new_visible_top)`. The topmost still-visible row, `new_visible_top`, is the one *next* frame's growth will scroll off. The union — `[prev_visible_top, new_visible_top]` — is the set of rows at risk. For every row in it, `whole_row` becomes true, forcing `x_first_diff` to `0`: the entire row is re-emitted from column 0, *even if the diff said it was unchanged.*

The comment states the rationale precisely:

> *"If prev_cells says 'matches canvas' but the wire actually shows old content (stale shadow, wide-char boundary mis-snap, layout shift between frames), that stale content commits. Defence: force a full-row re-emit for every row that this frame will scroll off, plus the topmost still-visible row."*

This is the belt to the shadow-invariant's braces. The shadow *should* always equal the wire. But scrollback corruption is so unrecoverable that the engine spends a few extra bytes to *re-assert* the correct bytes on precisely the rows about to become permanent — so that even if some upstream bug left the shadow stale for those rows, the wire is corrected in the last instant before it freezes. The history note records that an earlier version protected only the single topmost about-to-commit row; if the frame grew by more than one row in a frame, the lower about-to-commit rows still leaked. The union window closed that gap.

There is a second, orthogonal full-row trigger: `full_row`, auto-enabled under Zed's terminal (and forceable with `MAYA_COMPAT_REPAINT=1`). Some terminals — Zed's notably — mis-track mid-row cursor moves (cursor-forward / CHA), landing sub-span updates at the wrong column and tearing the frame. Under those terminals, every *changed* row is re-emitted whole from column 0 using only `\r` + content, never a mid-row horizontal move. The guard `&& row_first_diff < W` matters: it must not force-emit rows the diff calls identical, or a one-cell change would balloon into a whole-frame re-emit (the source cites a bench of 3137 bytes/frame versus 96 bytes/frame). Note the

difference in scope: the Zed compat repaint is about *horizontal* correctness on quirky terminals; the `will_scroll_off` repaint is about *scrollback-freeze* correctness on *every* terminal.

Chapter 7 — The incremental hash: proving the shadow cheaply

The shadow-of-wire invariant is only useful if the engine can *check* that the shadow has not been corrupted out-of-band before it trusts it for a diff. The naive check — re-read every cell of the shadow and compare — is `O(prev_rows × W)` per frame. On a 7,000-row transcript that is millions of cell reads every frame, and it was a real source of CPU on tall turns. The engine replaces it with an incrementally-maintained hash.

Two fields cooperate: `row_hashes_` holds one 64-bit hash per shadow row (an FNV-style fold of the row's cells, mixed with the row *index* so a reordering is detectable), and `shadow_hash_` is their XOR-fold:

```
inline uint64_t combine_rows(const std::vector<uint64_t>& rows) {
    uint64_t h = 0;
    for (uint64_t r : rows) h ^= r;
    return h;
}
```

The key property is that XOR is its own inverse, so the fold can be maintained incrementally. At the end of each compose, only the rows the frame actually changed need re-hashing. The engine XORs each changed row's *old* hash out of the fold, re-hashes the row, and XORs the *new* hash in:

```
uint64_t h = state.shadow_hash_;
for (int y = content_rows; y < prev_rows; ++y)   // shrink: fold out tail
    h ^= state.row_hashes_[y];
state.row_hashes_.resize(content_rows);
for (int y = begin; y < content_rows; ++y) {
    h ^= state.row_hashes_[y];                    // XOR OUT old
    uint64_t nh = hash_row(shadow_base + y*W, W, y);
    state.row_hashes_[y] = nh;
    h ^= nh;                                       // XOR IN new
}
state.shadow_hash_ = h;
```

By XOR-associativity the result is bit-identical to a full re-fold, at `O(changed rows)` cost instead of `O(content_rows)`. On a long streaming turn, where only the tail changes each frame, this collapses a per-frame linear cost to effectively constant.

The checker, `verify_shadow`, is the symmetric reader. On the hot path (release builds) it does not re-fold at all — the incremental maintenance already keeps `shadow_hash_` correct by construction — so it returns in `O(1)`:

```
#ifndef NDEBUG
    const uint64_t h = combine_rows(state.row_hashes_);
    if (h != state.shadow_hash_) return std::nullopt;   // poisoned
    return ShadowWitness{&state, h};
#else
    return ShadowWitness{&state, state.shadow_hash_};    // trust maintained hash
#endif
```

It also rejects the one realistic out-of-band corruption cheaply: if the row-hash array's size does not match `prev_rows_`, something cleared or resized it outside the compose path, and `verify_shadow` returns `std::nullopt` — "the shadow is poisoned; do not diff." That `nullopt` is the entry point to the recovery machine, and the return type

`std::optional<ShadowWitness>` is the beginning of the Witness Chain — the subject of the next three chapters.

Chapter 8 — The Witness Chain, part I: ShadowWitness

Everything so far is *discipline*: the diff maintains the shadow, the hash proves it, the caller is *supposed* to check the hash before diffing. But "supposed to" is exactly how bugs happen. The source is explicit that every scrollback bug in the codebase's history was a path that *forgot*, or short-circuited, the check. So the engine lifts the obligation out of discipline and into the type system. It is called the **Witness Chain**, and it makes "diff without proving the shadow matches the wire" a *compile error.*

The first link is `ShadowWitness`, declared in `serialize.hpp`. Its type annotation:

> *"ShadowWitness is single-use proof that the shadow matches the wire — dropping it forces the next render to re-verify or fall through to recovery."*

It is a move-only token with a *private* constructor. There is exactly one producer: `verify_shadow`. And the compose entrypoint *requires* a `ShadowWitness&&` as an argument. Chain the two facts together and the conclusion is airtight: the only way to obtain the argument compose demands is to run the verification that produces it. You cannot fabricate a witness — the default constructor is deleted:

```
ShadowWitness() = delete;
private:
    ShadowWitness(const InlineFrameState* s, std::uint64_t h) noexcept
        : state_(s), hash_at_issue_(h) {}
    friend std::optional<ShadowWitness>
        verify_shadow(const InlineFrameState&) noexcept;
```

Provenance binding. The witness carries a pointer to the exact state it was minted against, plus the hash value at issue time:

```
const InlineFrameState* state_;
std::uint64_t           hash_at_issue_;
```

When compose consumes the witness it re-derives the hash and compares against `hash_at_issue()`. This closes the window between *verify returned* and *compose ran.* If the cells were mutated in that span, the re-hash will not match. The response, from `frame_bytes.cpp`, is deliberately severe:

```
if (h != witness.hash_at_issue()) {
    // Shadow corruption between verify and consume.
    // Cannot emit bytes safely; abort the process.
    std::abort();
}
```

Why abort rather than recover? Because the renderer is single-threaded and the witness was taken immediately upstream. A mismatch here means the cell buffer changed between two consecutive function calls within one tick — which implies a thread-safety bug or a hardware fault. The source's reasoning: *continuing to run is worse than stopping.* Recovery is for *expected* divergence; this is *impossible* divergence, and the only honest response is to stop.

Single-use via move-only. The witness is move-only, and consumption moves it. A second consumption is a use-after-move, producing a witness with a null state pointer that fails the `valid()` assertion. Compilers warn on it under `-Wmoved-from`. So a witness cannot be reused across frames — each frame must mint a fresh proof. The type system enforces *freshness*, not just existence.

Chapter 9 — The Witness Chain, part II: ScrollbackProof

`ShadowWitness` proves the shadow matches the wire's *visible viewport.* It says nothing about the rows that already scrolled *off* into frozen scrollback. Those rows are the other half of the correctness theorem, and they get their own token: `ScrollbackProof`.

The problem it solves is specific. When a frame has overflowed (prev_rows > term_h some rows are in scrollback. The diff must never rewrite them. But there is a subtler hazard: what if the *content* of those committed rows in the new canvas differs from what physically scrolled off? That happens on a wholesale model swap (loading a different conversation), or if a layout shift moved a block. The diff, scanning only the visible range, would skip those rows — leaving the old bytes stranded in scrollback while the model believes they hold the new content. The result is a duplicated or mismatched prefix.

The guard is a memcmp of the canvas's top overflow rows against the shadow's committed prefix, `InlineFrameState::scrollback_prefix_matches`. Historically this was a bare `bool` the render path was *disciplined* to call. The type annotation on `ScrollbackProof` records what went wrong with that:

> *"Every scrollback-corruption bug in this codebase's history was a frame that reached the diff with a SHIFTED committed prefix because some path forgot, or short-circuited, that check. ScrollbackProof lifts the obligation into the type system, exactly as ShadowWitness did for the shadow."*

So, identical pattern: move-only token, private constructor, sole producer (check_scrollback consumed by value in `InlineFrame<Synced>::render`. "Render an overflowed frame without checking the committed prefix" is a compile error. There are two discharge shapes, both carried by the same type so the render signature is uniform:

Vacuous

The frame fits the viewport (prev_rows ≤ term_h Nothing overflowed, there is no committed prefix, the obligation is trivially true. `check_scrollback` returns a vacuous proof.

Witnessed

The frame overflowed *and* the committed prefix is byte-identical to what physically scrolled off. The memcmp ran and matched.

When the prefix has *shifted*, `check_scrollback` returns `std::nullopt`, and the caller must recover (commit the overflow, then soft-repaint) instead of rendering. This is the same control flow the old bool-gate expressed — now unforgeable.

The proof carries the same provenance binding as `ShadowWitness`: the address of the state it was minted against and the overflow row count it validated, so render can assert (in debug) that the proof matches the state it is rendering. Two witnesses, two halves of the theorem: the visible viewport and the frozen prefix. Together they cover every cell.

Chapter 10 — The Witness Chain, part III: FrameBytes and commit_to

---

The witnesses prove the *precondition*. The last link handles the *postcondition*: that the state advance and the byte delivery are *inseparable.* This is `FrameBytes`, a capsule holding the bytes to write and the successor state, returned by the witness-chain entrypoint `compose_inline_frame` in `frame_bytes.cpp`.

The hazard it closes is a torn commit. Suppose compose produced bytes and a new shadow, the code updated the shadow to reflect the new frame, and *then* the write to the terminal failed partway. Now the shadow describes a frame the wire never fully received. Every future diff runs against a lie. The `FrameBytes` capsule makes this impossible by *fusing* the two operations:

```
CommitOutcome FrameBytes::commit_to(Writer& writer) && noexcept {
    if (bytes_.empty())
        return commit::Synced{std::move(successor_)};

    Status st = writer.write_or_buffer(bytes_);
    if (st)
        return commit::Synced{std::move(successor_)};

    // Hard I/O error: drop the residue and surface HardReset.
    writer.discard_residue();
    return commit::HardReset{st};
}
```

The successor state is *only* handed out — wrapped in the `commit::Synced` arm — when the write actually succeeded. If the write fails hard, the in-flight successor is *dropped*, and the outcome is `commit::HardReset`: the wire is in an unknown state, so the next render will emit the full wipe-and-repaint. The source is explicit that dropping the successor is the point: *none of its invariants hold any more (prev_cells now reflects bytes* the wire never received)."

There are two more consumption shapes, each a typed transition:

`abandon()` → `commit::Stale`
: The bytes are discarded without writing (e.g. a higher-level decision not to send this frame). The wire still shows the prior frame, but the successor was already computed for a frame that never shipped. So the state is run through `abandoned_for_recovery()` and returned as `Stale` — the next render takes the case-(B) soft redraw, repainting in place against what the wire plausibly still shows.

`abandon_to_hard_reset(reason)` → `commit::HardReset`
: The runtime observed, from a parallel path, that the wire is genuinely unknown. Drop the bytes, surface `HardReset`.

Notice the shape of the whole chain now. `verify_shadow` returns `optional<ShadowWitness>`; `check_scrollback` returns `optional<ScrollbackProof>`; `compose_inline_frame` consumes both witnesses and returns `FrameBytes`; `FrameBytes::commit_to` consumes the bytes and returns a typed `CommitOutcome` that *is* the next coherence state. At no point can you obtain the next state except by discharging every proof and successfully writing the bytes. The corruption modes are not defended against at runtime — they are *unrepresentable*.

Chapter 11 — Coherence: the render-time state machine

The witnesses gate a single frame. The *coherence state machine* decides, each render, which kind of frame to produce. It lives in `Runtime` in `app.hpp` as a variant of typed states. For inline mode the states are:

```
inline_frame::InlineFrame<Empty>       // never rendered
inline_frame::InlineFrame<Fresh>       // first paint pending
inline_frame::InlineFrame<Synced>      // shadow matches wire
inline_frame::InlineFrame<Stale>       // needs soft redraw (case B)
inline_frame::InlineFrame<HardReset>   // needs wipe + repaint
inline_frame::InlineFrame<Sealed>      // finalized
```

The tag is a phantom type: the same underlying state, but the tag selects which `render` overload is legal, so illegal transitions are compile errors. The normal life of a frame is `Fresh` → `Synced` → `Synced` → `...`, each `Synced` render running the full Witness Chain and returning a new `Synced` via `commit::Synced`.

The interesting transitions are the recoveries, and they are exactly the `Cmd` handlers a host can dispatch:

```
void commit_inline_overflow() noexcept {
    ...
    const int overflow_rows = s->rows() - term_h;
    if (overflow_rows <= 0) return;
    in_coherence_ = std::move(*s).commit(
        s->scrollback_marker(overflow_rows));
}

void force_redraw() noexcept {
    fs_coherence_ = coherent::Divergent{};
    if (auto* s = get_if<InlineFrame<Synced>>(&in_coherence_))
        in_coherence_ = std::move(*s).demote_to_stale();
}

void reset_inline() noexcept {
    fs_coherence_ = coherent::Divergent{};
    if (auto* s = get_if<InlineFrame<Synced>>(&in_coherence_))
        in_coherence_ = std::move(*s).demote_to_hard_reset();
    else if (auto* st = get_if<InlineFrame<Stale>>(&in_coherence_))
        in_coherence_ = std::move(*st).escalate_to_hard_reset();
}
```

Three primitives, three severities. They are *not* interchangeable, and picking the wrong one is its own class of bug. That is important enough to get the whole next chapter.

One subtlety worth noting here: both `commit_inline_overflow` and its sibling `commit_inline_prefix` *shift the shadow while remaining Synced.* That is, they mutate `prev_rows_` and `prev_cells_` without changing the coherence *tag*. The render path has a canvas-preservation optimization gated on the coherence index, and these two structural events are invisible to it — so they set a one-shot inhibitor (scrollback_recovery_count_ to suppress that optimization for the frame after the shift. This is the kind of second-order interaction that makes the "just remain Synced" shortcut subtle, and it is documented at the field in `app.hpp`.

Chapter 12 — Recovery primitives: the chooser and its traps

Three host-callable primitives interact with the inline render state. The source is emphatic that they are not interchangeable. Here is the chooser, then the trap that motivates it.

commit_scrollback_overflow — bookkeeping only. Calls `commit_inline_overflow`, which advances `prev_cells_` by `max(0, prev_rows - term_h)` rows and recomputes the hash. **Zero bytes are written to the wire.** After it, `prev_rows ≤ term_h` and `updatable_start == 0`, so the next diff scans every visible row. It is safe because the rows dropped from the shadow were *already* emitted to the wire by earlier streaming and are already in native scrollback — the commit just acknowledges that fact. Use it for a bounded-frozen trim (dropping the oldest N entries from the transcript).

force_redraw — soft viewport repaint. Demotes `Synced` to `Stale`; the next compose enters case (B) — cursor walks up, every visible row is re-emitted in place, then `\x1b[J` erases below. Use it for ghost cells *inside* the live viewport, and for a Ctrl-L user redraw hotkey. But it carries a hazard: case (B)'s "new frame taller than the cursor's offset from viewport top" branch emits bottom-edge newlines to make room, and *each such newline permanently scrolls one row of host content into* scrollback." Under streaming that is correct (the rows scrolling off belong to the current turn). Under a *model swap* those rows are unrelated host history, and the scroll destroys them.

reset_inline — hard wipe, host-callable. Demotes to `HardReset`, whose next render emits `\x1b[2J\x1b[3J\x1b[H` — wipe viewport, wipe saved lines, home cursor — then repaints fresh. The `\x1b[3J` is the *only* host-callable escape that can reach off-viewport frozen rows, and it does so by destroying *all* of them, including pre-agentty shell history. It is reserved for an explicit, destructive, user-initiated content swap.

The trap: model swap into shorter content. This is worth internalizing because two "obvious" fixes are both wrong. When you switch to a different conversation (thread swap / new thread), the old thread may have overflowed, committing dozens of its rows to frozen scrollback. Now:

- **force_redraw** seems right — "my shadow no longer matches what the user should see, force a repaint." But case (B)'s bottom-edge newlines scroll the old thread's tail *and the host's shell history* into oblivion. Wrong.
- **commit_scrollback_overflow** seems right — it advances `prev_rows` down to `term_h` and fixes the mid-viewport seam the diff would otherwise leave. But it writes *zero bytes*, so the old thread's rows already in frozen scrollback *stay physically on the wire.* If the new thread is *shorter*, the next frame shrinks within the viewport and leaves a copy of the old transcript stranded above the new one. Wrong — and intermittent, because it only bites when the new content is shorter than the old overflow.

Only `reset_inline` is correct here: its `\x1b[3J` is the sole primitive that reaches those stranded off-viewport rows. The source's comment at `reset_inline` in `app.hpp` states it plainly — neither `force_redraw` nor `commit_scrollback_overflow` "can erase those rows; they strand a copy of the old transcript above the new one. demote_to_hard_reset is the only coherent fix." And it adds the guardrail: *Do NOT wire this to a per-frame or per-turn path* — the scrollback wipe is acceptable *only* for an explicit user-initiated swap.

This is why agentty's `picker.cpp`, handling `ThreadListSelect` and `NewThread`, dispatches `reset_inline` and nothing else. The chooser table, from the source:

```
Ghost cells inside the live viewport   -> force_redraw
Ctrl-L user redraw hotkey              -> force_redraw
Wholesale model swap (thread switch)   -> reset_inline
Bounded-frozen trim (drop oldest N)    -> commit_scrollback_overflow
Terminal genuinely corrupted (external) -> (none - resize only)
```

Chapter 13 — agentty as a client: trim, swap, and the ledger

---

maya provides the mechanism; agentty is the policy layer that drives it correctly. Three interactions are worth studying because they show the chooser from Chapter 12 in real use.

1. Submit is not a swap. When you send a message, `submit_message` in `modal.cpp` appends to the existing transcript. It deliberately does *not* call `commit_scrollback_overflow`:

> *"Submit is not a wholesale model swap — it appends to the existing transcript, so maya's normal row diff handles the composer-shrink + new-turn-rows transition correctly. Past versions fired commit_scrollback_overflow on every submit to flush a suspected composer-shrink seam, but the call advances prev_cells past whatever the renderer thinks has overflowed."*

The lesson: reach for a recovery primitive only when the normal diff genuinely cannot handle the transition. Appending is the diff's home turf; intervening only creates drift.

2. Trim fires commit_scrollback. When the transcript grows past a soft cap, agentty freezes and drops the oldest entries. `trim_frozen_if_oversized` in `frozen.cpp` computes exactly how many rows it dropped and returns a `Cmd::commit_scrollback(N)` — the row-counted commit that shifts the shadow to match. The header comment records the whole history of this being the drift-prone seam:

> *"[the trim pipeline] kept in lockstep across four parallel vectors, and summed into the exact commit_scrollback count each trim sent maya. Every historical trim-corruption bug was drift between that pipeline and what maya actually painted."*

The word *exact* is the whole point. The commit must drop *precisely* the rows that were rendered and scrolled off — not an estimate. Estimating this count (from a reconstructed row height) was the source of the classic "phone-over-SSH duplication ghost," which brings us to the third interaction.

3. The ledger and seal_measured. The deepest fix in the tree moves the *measurement* of a frozen block's height out of the host and into maya. Previously agentty reconstructed the width a block would wrap at — "terminal columns minus the chrome paddings I believe wrap my fragment" — and any mistake in that arithmetic (the famous `-2 vs -4` padding-stack comment) made wrapped blocks under-measure on narrow terminals, producing a duplicated ghost. The `ScrollbackLedger::seal_measured` function in `scrollback_ledger.cpp` eliminates the reconstruction entirely:

> *"the paint pass records the ACTUAL width constraint compute() handed the ledger's fragment (record_paint_width), and seal_measured lays the new block out at exactly that width. There is nothing left for the host to reconstruct; the chrome can change shape without any host code knowing."*

This is the same philosophy as the Witness Chain, applied one level up: do not let the host *estimate* a quantity that must be *exact*. Make the party that *knows* the true value (the renderer, which actually performed the layout) record it, and have everyone else read it. The ledger's `seal_measured` runs a real layout pass at the paint-recorded width, inside a scoped render context pinned to that width, so the freeze frame lays out at the true height and the seam is byte-stable — no gate recovery, no

ghost.

Chapter 14 — Epilogue: why this shape, and what it buys

Step back from the mechanism and look at the shape of the whole design, because the shape is the real lesson.

The problem was: a program must maintain a perfect model of an external, un-readable, partly-immutable device, and any drift between model and device is permanent and user-visible. The solution has four strata, each a different kind of guarantee:

1. A single linear model.
`InlineFrameState` is move-only so there is exactly one shadow of the one wire. No parallel models can disagree.

2. An inductive maintenance invariant.
The diff writes every emitted cell back into the shadow, so shadow equals wire by induction over frames. The `will_scroll_off` defence re-asserts the correct bytes on precisely the rows about to become permanent — spending a few bytes to make the one irreversible moment safe.

3. A cheap continuous proof.
The incremental XOR-fold hash lets the engine *verify* the model in `O(1)` on the hot path, so verification is affordable enough to do every frame.

4. Type-system enforcement.
The Witness Chain — `ShadowWitness`, `ScrollbackProof`, `FrameBytes` — turns "you must verify before you diff" and "state advance must be atomic with byte delivery" from *discipline* into *compile errors.* The corruption modes are not caught at runtime; they are *unrepresentable*.

That progression — from a data-structure invariant, to a runtime proof, to a type-level impossibility — is the template. The comments in the source keep returning to the same confession: *every historical corruption bug was a path that forgot the check.* The Witness Chain is the answer to that confession. You cannot forget a check that the compiler will not let you skip.

And the recovery machine is the humility layer. Not every divergence can be prevented — an external process can write to the tty, an SSH link can drop bytes, a terminal can mangle a resize. So the engine has a small, sharp set of recovery primitives, each matched to exactly one class of divergence, with a documented trap for the wrong choice. The severity ladder — bookkeeping commit, soft viewport repaint, hard wipe — maps onto how much of the wire's state has become unknown. The cheapest recovery that restores correctness is always the right one, and the source spells out, primitive by primitive, when each applies.

What all this buys is a single, quiet property: you can stream a thousand-row AI response into a shared terminal, scroll up through your whole interleaved session, resize your window, switch conversations, and never once see a duplicated composer, a drifting prompt, or a half-frozen card. The frozen row is always correct, because the instant before it froze, the engine had proven — not hoped — that it was.

— fin —

Colophon

This book was written directly from the maya and agentty source trees. Every quoted comment, function signature, field name, and escape sequence appears in the code as shown; where prose in the repository's own design docs disagreed with the code, the code was taken as authoritative. The primary sources are:

```
maya/include/maya/render/serialize.hpp
maya/src/render/serialize.cpp
maya/src/render/frame_bytes.cpp
maya/src/render/scrollback_ledger.cpp
maya/include/maya/app/app.hpp
agentty/src/runtime/app/update/picker.cpp
agentty/src/runtime/app/update/frozen.cpp
agentty/src/runtime/app/update/modal.cpp
agentty/src/runtime/app/update/meta.cpp
```

Typeset with groff (`ms` macros), rendered to PDF via `gropdf`.